Software Architecture

Submitted by
Andrea Gentilini (880141@stud.unive.it)
Enrico Miatto (882816@stud.unive.it)
Simone Biondo (879899@stud.unive.it)

Project Name
ISTL – Italian Supreme Tennis League

Università Ca’ Foscari Venezia
Year 2025/2026

Summary

1. Introduction

The purpose of this document is to describe the architectural design and the main technological decisions adopted for the development of **ISTL (Italian Supreme Tennis League)**, a web-based system designed to manage racket-based sport tournaments such as tennis, padel, and table tennis.

The project has been developed as part of the *Software Architectures* course and is based on a predefined **architecture kata**, which specifies functional requirements, non-functional requirements, constraints, and a prioritized set of architectural characteristics (e.g. modularity, maintainability, elasticity).

The report focuses on:

- A brief description of the selected technologies
- How the system requirements have been translated into architectural decisions
- A comparison between two architectural samples:
    - A **monolithic architecture**
    - A **distributed architecture**

2. Problem Statement and Context

The ISTL system aims to unify and simplify the management of sport tournaments by providing a single platform capable of handling the entire tournament lifecycle, from player registration to match results and rankings.

The system targets a heterogeneous user base, including:

- Tournament organizers
- Referees
- Final users (players and spectators)

Several **constraints and assumptions** strongly influence the architecture:

- Limited infrastructure budget
- Limited operational support in case of downtime
- No real-time requirements for score updates
- No handling of online payments or sensitive medical data
- Potential future integration with external systems, currently out of scope

Given these constraints, the system favors simplicity, maintainability, and cost efficiency over highly complex or cutting-edge architectural solutions.

3. Technology Stack

ISTL is implemented as a **web-based application** following a classic client–server model.

At a high level, the system consists of:

- A web frontend responsible for user interaction
- A backend application exposing business logic and APIs
- A relational database used for persistent data storage

The core technologies adopted are listed in the following table. Underlining the key differences between the two samples (monolithic and distributed).

	Monolith	**Distributed**
Backend	Ruby on rails	
Frontend	Ruby on rails	React with Typescript
Database	PostgreSQL *Hosted on a Virtual Machine in Amazon Web Services*	
Proxy / API Gateway	-	NGINX Proxy Manager
Containerization	2 Docker containers	4 Docker containers

3.1. Frontend

Frontend is the layer that changes the most between the two samples. In the following pages we present a list of the most important technologies we used to develop the frontend layers.

Ruby ERB

ERB is a server-side templating engine used by Rails to generate dynamic HTML. It allows Ruby code to be embedded directly into HTML templates, enabling data-driven page rendering but resulting in tight coupling between presentation logic and backend code.

React

React provides a component-based approach to user interface development, where the UI is composed of small, self-contained components that encapsulate structure and behavior. Compared to ERB templates, React components promote clearer separation between presentation and logic, improving maintainability and readability.

From a practical standpoint, React also aligns well with a more interactive and client-driven user experience, allowing state and UI updates to be handled entirely on the client side without continuous server-side rendering.

TypeScript

TypeScript adds static typing to JavaScript, which significantly improves code robustness and developer experience. In the context of a distributed architecture, this is particularly valuable, as frontend code relies on data contracts defined by the backend. TypeScript helps ensure that frontend expectations are aligned with the GraphQL schema and that mismatches are detected early during development.

Vite

Vite is used as the build and development tool for the frontend. Its main advantage lies in fast startup times and efficient hot module replacement, which shorten development feedback loops. This was especially beneficial given the iterative nature of frontend development, allowing changes to be tested and validated quickly without complex configuration or long rebuild times.

GraphQL (and why we didn’t choose a simple Rest API)

Communication between the frontend and backend in the distributed architecture is implemented using GraphQL instead of traditional REST APIs. From a frontend development perspective, this choice proved to be particularly effective and aligned well with the goal of frontend–backend decoupling.

GraphQL exposes a strongly typed schema that defines all available queries and mutations, acting as a clear and explicit contract between the two sides. This allows the frontend to request exactly the data it needs, in the required shape, without depending on multiple endpoints or server-driven response formats. As a result, frontend code becomes more concise and easier to maintain.

Compared to REST APIs, GraphQL introduces additional complexity on the backend, including schema design and resolver management, and may require careful performance tuning. However, these drawbacks are balanced by the flexibility and clarity it provides at the interface level.

3.2. Backend

The backend instead is basically the same between the two architectures, the backend is implemented using Ruby on Rails. The following list shows the most important features (Gems, in Ruby's slang) we used.

Ruby on Rails

Ruby on Rails is a full-stack web framework that follows the Model–View–Controller (MVC) pattern. It provides built-in support for routing, database access via its ORM capabilities, authentication integration, and testing, allowing rapid development while maintaining a clear separation of concerns.

Devise & Devise Token Auth

Devise is used to handle authentication and authorization within the application, providing support for user registration, login, and role management. Devise Token Auth extends this mechanism to support token-based authentication, making it suitable for API-based and distributed deployments.

Active Admin

Used to quickly build a web-based administrative interface. It allows administrators to manage core domain entities through a structured UI, reducing the need to implement custom back-office functionality.

RuboCop

Gem that implements static analysis and a linting tool for Ruby. It enforces consistent coding conventions, detects potential code smells, and helps maintain code quality across the project.

3.3. Proxy / API Gateway

NGINX Proxy Manager is used as a reverse proxy acting as a **simple API gateway** between clients and backend/frontend services. It centralizes request routing, exposes backend APIs through a single entry point, and decouples clients from internal service locations. While simpler than a full API gateway solution, it effectively supports distributed deployment, basic access control, and future scalability needs without adding significant operational complexity.

3.4. Database

ISTL uses **PostgreSQL** as its relational database management system, and its data layer is intentionally kept simple.

The database plays a central role in the stability and correctness of the system. As a consequence, changes to the database structure must be carefully managed. Sharing a single database schema creates tight coupling between services.

Rails Migrations

To mitigate the risks of having **improper or insecure data management**, all database schema evolutions in ISTL are handled through Rails migrations. Rails migrations provide a structured way to define and apply changes to the database schema over time. Each migration represents a small and explicit modification. These migrations are stored as part of the application source code and are versioned together with it. This ensures that the database schema evolves in a controlled and traceable way, aligned with the evolution of the application itself.

One important benefit of using migrations is consistency across development environments. Because the schema is defined by a sequence of migrations, any developer can recreate the expected database structure by applying the same migration history. This avoids problems caused by manual or undocumented changes and reduces the risk of differences between local development setups.

Rails migrations also support **repeatability and automation**. The framework keeps track of which migrations have already been applied, ensuring that each change is executed only once and in the correct order. This makes it easier to integrate database changes into deployment and testing workflows, where schema updates can be applied automatically and reliably.

Ngrok

Ngrok is a tunneling service that securely exposes a local or private service to the public internet. It creates a temporary public URL that forwards external traffic to a specified local port, without requiring firewall or network configuration changes.

4. Monolithic Architecture Sample

4.1. Architectural Overview

The first sample adopts a **layered monolithic architecture**. All application logic is deployed as a **single unit**, following a classic **n-tier structure** (in our case n = 3) where responsibilities are separated at a logical level rather than at deployment time.

In this sample:

- Presentation, business logic, and data persistence are organized in layers
- Communication between components happens through in-process method calls
- The system is perceived and deployed as a single block, despite internal structure

To simplify development and testing, the database container is hosted on a **cloud-based virtual machine in AWS**. The database is then exposed through **ngrok**, which provides secure tunneling and allows external access to the PostgreSQL service.

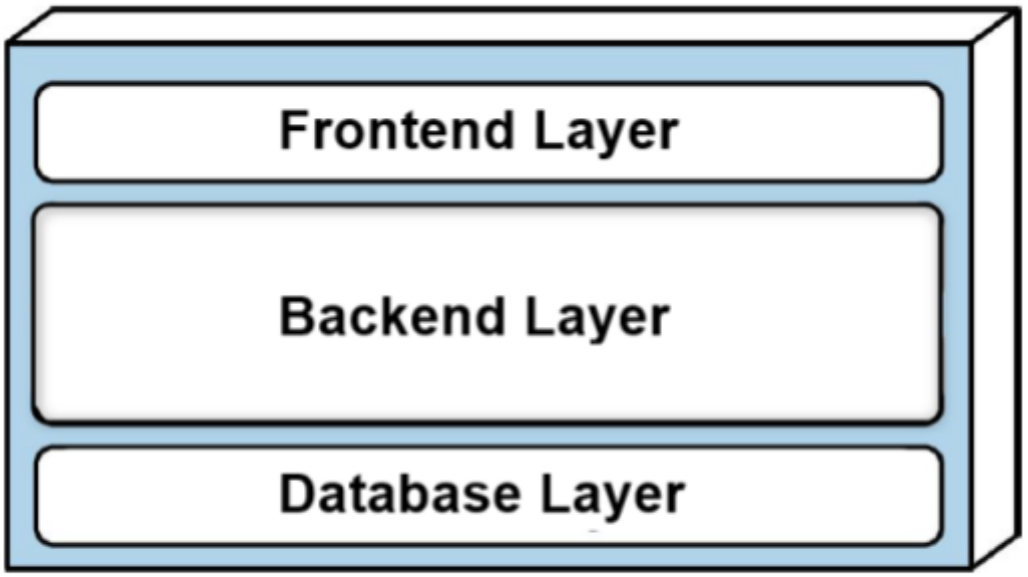

Figure 1: Architecture representation of Monolithic Layered

4.2. Rationale

This architectural choice prioritizes **simplicity, low deployment complexity, and fast development**, which aligns well with the early stage of the project and the limited infrastructure constraints.

As discussed in the lectures, layered monoliths are a common and effective solution for small to medium web applications, but they inherently limit scalability and independent evolution of components.

The choice of separating the database was a practical decision mainly driven by development and testing needs. This setup allows multiple developers to connect to the same database instance and work with a shared dataset, ensuring consistent test data across different environments while still keeping the main application logic centralized in a single deployable unit.

4.3. Internal Structure

Internally, the Rails application is organized according to the MVC pattern.

- **Models** are responsible for representing the domain concepts and managing data access, including interactions with the PostgreSQL database.
- **Controllers** handle incoming HTTP requests, coordinate the execution of application logic, and prepare data for the views.
- **Views** are responsible for presentation logic and define how information is rendered and delivered to the user.

This internal structure enforces a clear separation of concerns, which improves readability and maintainability of the codebase, even though the application is deployed as a single monolithic unit.

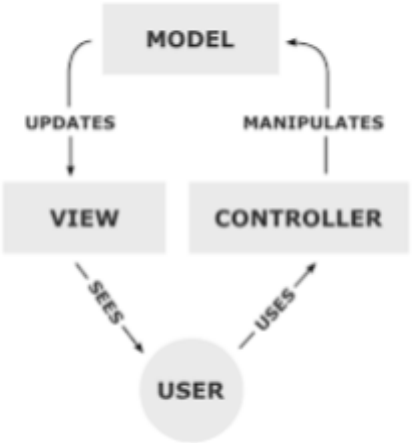

Figure 2: Workflow of a MVC application

4.3.1. Frontend development - Monolithic

In the monolithic architecture, the frontend is implemented using Ruby on Rails through ERB (Embedded Ruby) templates. While this approach is integrated into Rails, it results in a strong coupling between presentation logic and backend concerns.
During development, **several issues emerged**. ERB mixes HTML and Ruby code, reducing readability and making templates harder to maintain. Additionally, since ERB and Rails conventions were new in this context, extra effort was required to understand syntax, framework behavior, and data flow. Developing frontend features required constantly checking controllers and models to identify where data originated and how it could be adapted for rendering or frontend logic.

As a result, frontend development was strongly dependent on backend structure and knowledge. The frontend could not be developed independently, leading to slower iteration and reduced flexibility, especially for developers more accustomed to client-side development.

4.3.2. Backend development - Monolithic

The backend will be basically the same in both the monolithic and distributed samples. The internal structure does not change; only the **deployment model** does.
In the monolithic sample, backend and frontend are deployed in the same container and requests are handled internally by the application.

5. Distributed Architecture Sample

5.1. Architectural Overview

The second sample moves toward a **distributed architecture**, in line with the concepts presented in the lectures on distributed systems. In this case, the system is composed of **multiple independently deployed components** that communicate over the network. The frontend and the backend are packaged into different Docker containers, each with its own runtime environment and lifecycle.

While the system does not fully adopt advanced distributed patterns (e.g. microservices or event-driven communication), this sample represents a **first step away from the monolithic model**.

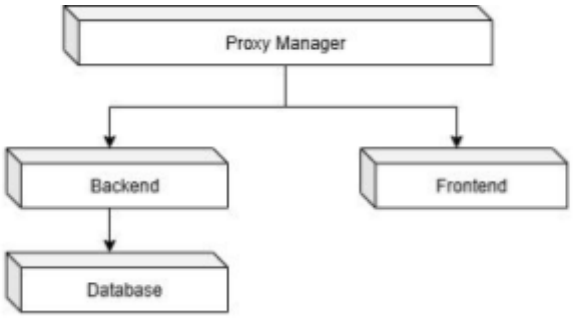

Figure 3: ISTL distributed architecture schema

5.2. Rationale

The primary motivation behind this distributed architectural sample is to explore the benefits of **increased modularity and separation of responsibilities**, while preserving the same functional behavior offered by the monolithic solution. By distributing the system into independently deployable components, the architecture aligns more closely with the principles of service-based and distributed architectures discussed during the course, particularly in terms of deployability, scalability, and evolutionary design.

Separating the frontend from the backend allows each component to be developed, tested, and deployed independently. This improves the overall deployability of the system, as changes to the user interface do not require redeploying the backend services, and vice versa. Moreover, this separation supports team autonomy, enabling frontend and backend developers to work in parallel with reduced coordination overhead.

The introduction of GraphQL as the communication mechanism further supports this goal by decoupling the data contract from the internal implementation details of the backend. Unlike REST APIs, where endpoints often reflect server-side resource structures, GraphQL shifts part of the responsibility for data shaping to the client. This results in a more flexible and expressive interface, which is particularly beneficial in distributed systems where frontend requirements may evolve rapidly.

Overall, this architectural choice represents a trade-off. While the system becomes more complex in terms of deployment and infrastructure management, it gains significant advantages in terms of modularity, maintainability, and long-term evolvability. In line with the course's emphasis on architectural trade-offs, this solution does not aim to be universally "better" than the monolithic one, but rather more suitable for scenarios where independent evolution and scalability are key concerns.

5.3. Internal Structure

Internally, the distributed architecture is organized around clearly defined responsibilities and well-established communication boundaries.

From an architectural standpoint, the following structure emphasizes loose coupling and high cohesion. Each container focuses on a well-defined set of responsibilities, and interactions between components are explicitly defined through standardized interfaces.

5.3.1. Frontend development - Distributed

In this sample, the frontend is implemented as an independent application and it follows a component-based design that promotes reuse, readability, and type safety.
Furthermore, the use of Vite as a build tool ensures **fast development cycles and efficient asset bundling**, which is particularly valuable in modern frontend development.

The React architecture emphasizes **high cohesion**, as each component encapsulates its own state, logic, and rendering responsibilities, and **low coupling**, achieved through clear module boundaries and explicit imports (static connascence). This design improves maintainability and limits the impact of changes across the codebase.

The frontend is fully decoupled from the backend and operates as a client-side rendered application. As a result, users can interact with the interface immediately after the initial load, regardless of backend performance, since pages are not generated server-side and navigation does not depend on backend-rendered views.

5.3.2. Backend development - Distributed

As stated on previous chapter, the backend between the two samples is basically the same, the only differences here are:

- The backend runs in a **separate Docker container** and is exposed through NGINX Proxy Manager (more info on the following sub-chapter), which acts as a reverse proxy and routes requests between frontend and backend.
- Its functionality (APIs) are exposed through a **GraphQL schema**, which defines the available queries and mutations as a formal contract between the backend and its consumers. Internally, the backend translates these GraphQL operations into domain-level actions and database queries against PostgreSQL.

5.3.3. Proxy Manager

In the distributed sample, a proxy server is used to route requests to the appropriate services through **NGINX Proxy Manager**.

In this setup, NGINX Proxy Manager fulfills an **API gateway–like role**, managing communication between distributed components:

- istl.distributed-backend.local - for backend service
- istl.distributed-frontend.local - for frontend service

It provides a single access point for the frontend, shielding backend services from direct exposure and simplifying client configuration. This approach improves security and prepares the system for future extensions, such as the introduction of additional services or alternative clients.

6. Architectural Characteristics and Design Decisions

This section describes how the architectural characteristics identified in the kata have been addressed in the ISTL project. For each characteristic, the rationale and the adopted implementation strategies are presented.

6.1. Simplicity & Cost

Given the limited development and support resources, simplicity is a key requirement. The system must be easy to understand, deploy, and maintain.

Implementation in ISTL

- Adoption of a monolithic architecture as the initial solution
- Use of Ruby on Rails, which enforces clear conventions
- Avoidance of unnecessary infrastructure components (e.g., service meshes, message brokers)
- Preference for synchronous, request–response communication
- Use of specific ruby gems that allows to have features already covered (authentication, admin operations, …)

6.2. Modularity

The high level of modularity offered by the distributed sample helps increase the maintainability of the system and keep it simple overall. A more distributed architecture allows individual components to be developed, tested, and modified independently, lowering the overall complexity of the system.

Implementation in ISTL
The ISTL project explores modularity through two architectural samples that reflect different design trade-offs.

- **Monolithic sample**
  The first sample exhibits a **limited level of modularity**. Front-end and back-end logic are deployed together within a single Docker container, and all application concerns are managed within the same codebase.
  Modularity in this sample is primarily **logical rather than physical**, relying on the internal structure imposed by the Ruby on Rails MVC pattern.

- **Distributed sample**
  The second sample is explicitly designed **with modularity as a primary goal**. In this architecture: Front-end and back-end are separated into distinct Docker containers, communication between components occurs through well-defined APIs and an API gateway acts as an integration point, enforcing clear boundaries between modules. This separation introduces **physical modularity**, allowing each component to evolve, scale, and be deployed independently. Although the database remains a shared component, the overall structure significantly reduces coupling compared to the monolithic sample.

As **future work**, modularity could be further improved by extending the system to support additional sports beyond racket-based disciplines. Different sports introduce specific rules, scoring systems, and tournament structures, which could be encapsulated into **dedicated modules**, each responsible for a specific sport type. This approach would limit the impact of changes, allow independent evolution of sport-specific logic, and make the system easier to extend without affecting existing functionality.

6.3. Maintainability

The system may evolve over time, but support resources are limited.For this scope, changes should be localized and low-risk.

Implementation in ISTL

- MVC pattern grants a lower level of coupling between elements (there is no interdependence between different controllers/models/views), and high cohesion (a view, a controller and a model work together for the final result).
- Centralized dependency management via **Gemfile**, **Gemfile.lock** and **NPM**
- Database schema evolution handled via **Rails migrations**

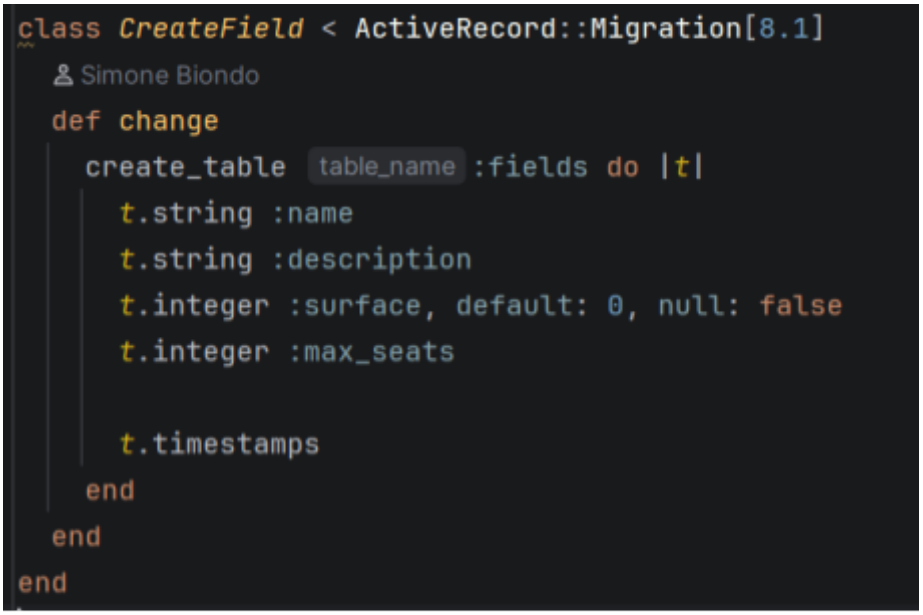

Figure 4: Example of ActiveRecord migration class

6.4. Testability

While full test coverage is not required and not specifically declared on the kata, critical functionality must be tested to reduce the risk of deploying broken code, especially given limited downtime support.

Implementation in ISTL

The ISTL project adopts a pragmatic testing strategy based on the Ruby on Rails default testing framework and a small, well-defined set of supporting libraries:

- **Minitest** allows writing unit and integration tests without introducing additional complexity or dependencies.
- **Factory Bot** is used to define reusable and consistent test fixtures. This improves test readability, reduces duplication, and makes tests easier to maintain as the domain model evolves.
- **Faker** is used in conjunction with Factory Bot to generate realistic but fake data (e.g., names, email addresses, tournament titles). This helps simulate real-world scenarios while avoiding hard-coded values and improving the robustness of tests.

```ruby
FactoryBot.define do
  factory :user do
    first_name { Faker::Name.first_name }
    last_name  { Faker::Name.last_name }
    birthdate { Faker::Date.birthday(min_age: 18, max_age: 65) }
    password { Faker::Internet.password(min_length: 8) }
    email { Faker::Internet.unique.email }
    uid { email }
    provider { 'email' }
  end
end
```

Figure 5: Test for "user" class using FactoryBot and Faker gems

6.5. Scalability & Elasticity

The system is expected to handle thousands of users, with peaks during tournament announcements and registration periods. It must be able to scale to accommodate temporary increases in load without requiring a complete architectural redesign.

Scalability is therefore considered a medium-priority characteristic, with an emphasis on horizontal scaling rather than vertical scaling.

Implementation in ISTL

Scalability in ISTL is primarily addressed through the use of **Docker containers**, which was selected as a foundational architectural choice to enable future scaling strategies with minimal changes to the application code.

- **Containerized application components**
  The backend application is packaged as a Docker container, ensuring that each instance of the application runs in an isolated and reproducible environment. This makes it possible to deploy multiple identical instances of the backend when increased load requires it.

- **Stateless application design**
  The backend is designed to be stateless, with all persistent data stored in an external PostgreSQL database. This allows multiple application containers to run concurrently without requiring session affinity or shared in-memory state, which is a prerequisite for effective horizontal scaling.

- **Readiness for horizontal scaling**
  By relying on Docker containers, the system can be scaled horizontally by increasing or decreasing the number of running containers based on demand. This approach supports both simple scaling strategies (e.g., manually starting additional containers) and more advanced orchestration-based solutions (more below).

- **[Future work] Compatibility with container orchestration platforms**
  The chosen container-based architecture allows ISTL to be easily integrated with orchestration platforms such as **Kubernetes or Docker Swarm**. These tools can manage container replication, health checks, and automatic scaling (e.g., adding or removing pods) based on resource utilization or predefined policies.

6.6. Deployability

Continuous deployment without downtime is not required. A simple and reliable deployment process is sufficient. Docker-based deployment ensures consistent environments

Implementation in ISTL

- The monolithic sample shows a **high level of deployability** due to its simplicity. The application is deployed using a small number of Docker containers
- The distributed sample, while offering architectural benefits, is inherently **more complex to deploy**. The increased number of containers introduces additional configuration, networking, and coordination requirements. As a result, deployment becomes more error-prone and requires a higher level of infrastructure management.
- **[Future work]** Deployability could be significantly improved by introducing **CI/CD pipelines**. Automated pipelines would allow consistent builds, testing, and deployments across environments.

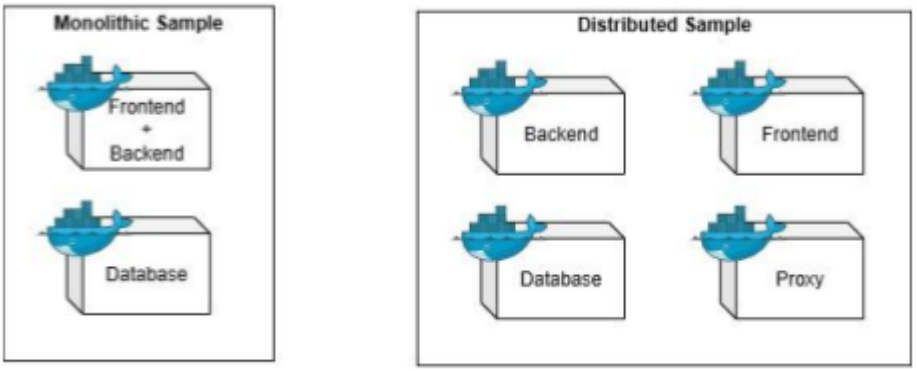

Figure 6: Deployed objects representation

6.7. Fault Tolerance

While the system does not implement advanced fault-tolerance mechanisms such as replication or automated failover, these choices are consistent with the project constraints. AsWhile the system does not implement advanced fault-tolerance mechanisms such as replication or automated failover, these choices are consistent with the project constraints. As **future work**, fault tolerance could be enhanced by introducing database replication, health checks (through log/metrics monitoring, e.g. Prometheus), and automated recovery mechanisms, especially in the distributed architecture.

Implementation in ISTL

- The system is designed to **fail safe** by preventing partial or inconsistent operations from being persisted. Critical operations that modify domain data are handled through controlled workflows, ensuring that failures do not leave the system in an invalid state.
- **Fail-secure** principles are also applied by enforcing authentication and authorization checks, ensuring that errors or unexpected conditions do not result in unauthorized access or privilege escalation.

6.8. Evolvability

Future integration with external systems is possible but currently out of scope.

In the previous chapters we listed (with “**Future work**” label) a few possible evolutions of our platform.

7. Conclusions

This report presented the architectural design of ISTL, highlighting how system requirements, constraints, and architectural characteristics guided the final decisions.

The monolithic architecture provides a solid and simple foundation aligned with the project context, while the distributed sample explores alternative trade-offs in terms of separation and extensibility.

For clarity reasons we would like to state that during the development of the project and the preparation of this report, artificial intelligence tools were used as a supporting aid. All design decisions and implementations remained under the responsibility of the development team.

7.1. Comparative development experience

From a development perspective, working on the distributed frontend was significantly more comfortable and aligned with existing skills. The clear separation between frontend and backend made it possible to treat the backend as a black box: only the GraphQL interfaces were required, and no knowledge of internal implementation details was necessary.

In contrast, frontend development in the monolithic architecture required constant interaction with backend files. Since ERB and Ruby were new technologies, understanding syntax, page structure, and data flow took additional time. Implementing dynamic rendering or frontend logic often meant inspecting models and controllers to determine how to fetch or adapt the required data.

The migration from the monolithic architecture to the distributed one was relatively straightforward precisely because of this decoupling. Backend logic could be reused, while the frontend was reimplemented as an independent application consuming well-defined interfaces.

Overall, the distributed approach better supports separation of concerns, independent development, and long-term evolvability. While it introduces additional architectural and infrastructural complexity, it represents a conscious trade-off that aligns well with modern software architecture principles and with the goals of this project.